

FeatherFlow: Poultry Management & E-Commerce System

Project Title:	FeatherFlow Poultry Management & E-Commerce System
Role:	Lead Systems Analyst (Database & Architecture Specification)
Target Audience:	Administrator (Farmer) & Customers (E-Commerce Buyers)
Technology Stack:	HTML5, CSS3, JavaScript, PHP (Backend), MySQL (RDBMS)

Executive Overview

FeatherFlow is a modernized, end-to-end relational database and web application designed to bridge the operational gap in local poultry farming businesses. It provides dual-facing functionality: an administrative dashboard for the poultry farmer (the Admin) to track flock cycles, egg collection, feed supplies, health logs, and financial transactions, alongside a highly responsive e-commerce storefront for retail and wholesale customers to browse, order, and pay for fresh poultry products online.

Phase 1: Business Requirements Specification

A. System Narrative & Scope

The poultry farm requires a secure, integrated system to manage internal agricultural logistics while facilitating external direct-to-consumer digital sales. The system operates on two core user pathways:

- Administrative (Farmer Portal):** The farmer must be able to track biological assets (poultry flocks divided by breed, age, and location), daily egg production logs, inventory cycles (feed stocks and supplements), health/ vaccination histories, and incoming web orders.
- Client-Facing (E-Commerce Storefront):** Registered customers can view product availability (e.g., live birds, dressed chickens, crates of eggs), place orders, choose pickup or delivery schedules, and check order fulfillment statuses.

B. Core Business Rules

- Flock Tracking:** A Flock is identified by a unique ID, breed, hatch date, and active bird count. Flock mortality or reductions must be logged to keep the headcount mathematically updated.
- Egg Production:** Eggs are logged daily per Flock. Logs must specify the quantity of healthy, cracked, or undersized eggs to compute yield efficiency.
- Product Inventory:** The e-commerce stock must automatically sync with real-time poultry batches and egg collections declared available for sale by the Admin.

4. **Order Fulfillment:** Customers can place an Order which consists of one or more OrderItems. An order cannot be finalized without a valid customer profile. Upon order payment, inventory is decremented.

Phase 2: Entity Types & Attribute Identification

To accurately capture the system's operational scope, the following fundamental tables have been identified:

- **Users (Admin & Customers):** UserID, Name, Email, PasswordHash, Phone, Address, Role (Admin / Customer), CreatedAt.
- **Flocks (Poultry Batches):** FlockID, Breed, HatchDate, InitialCount, CurrentCount, Status (Brooding, Laying, Harvested).
- **Egg Logs (Daily Yield):** LogID, FlockID, LogDate, GoodQuantity, DamagedQuantity.
- **Products (For Sale):** ProductID, ProductName, Category (Meat, Eggs, Live, Fertilizer), UnitPrice, StockQuantity, Description.
- **Orders:** OrderID, CustomerID, OrderDate, TotalAmount, Status (Pending, Processing, Shipped, Completed, Cancelled).
- **OrderItems (Line Items):** OrderItemID, OrderID, ProductID, Quantity, Subtotal.

Phase 3: Data Integrity & Constraints Specification

Every attribute must conform to precise technical types and operational constraints to guarantee high referential integrity.

Table Name	Attribute	Data Type	Primary/Foreign Key	Nullability / Domain Constraints
Users	UserID	INT AUTO_INC	Primary Key	NOT NULL
	Email	VARCHAR(100)	-	NOT NULL, UNIQUE
	Role	VARCHAR(15)	-	NOT NULL, CHECK (Role IN ('Admin', 'Customer'))
Flocks	FlockID	INT AUTO_INC	Primary Key	NOT NULL
	CurrentCount	INT	-	NOT NULL, CHECK (CurrentCount >= 0)
EggLogs	LogID	INT AUTO_INC	Primary Key	NOT NULL
	FlockID	INT	Foreign Key (Flocks)	NOT NULL, ON DELETE CASCADE
Products	ProductID	INT AUTO_INC	Primary Key	NOT NULL
	UnitPrice	DECIMAL(10,2)	-	NOT NULL, CHECK (UnitPrice >= 0.00)
Orders	OrderID	INT AUTO_INC	Primary Key	NOT NULL
	CustomerID	INT	Foreign Key (Users)	NOT NULL
OrderItems	OrderItemID	INT AUTO_INC	Primary Key	NOT NULL
	OrderID	INT	Foreign Key (Orders)	NOT NULL, ON DELETE CASCADE
	ProductID	INT	Foreign Key (Products)	NOT NULL

Phase 4 & 5: Entity Relationship Outline & Cardinality

The structural connections between our data assets are mapped below to aid frontend and backend database engineering:

- **Users to Orders (1:M):** One Customer can place multiple orders over time. An order is assigned to exactly one Customer.
- **Orders to OrderItems (1:M):** Each Order contains one or more item lines. Deleting an Order deletes its associated OrderItems (Cascade).

- **Products to OrderItems (1:M):** A catalog Product can appear as a line item across multiple transactions. Deleting a product is restricted if historical transaction records exist.
- **Flocks to EggLogs (1:M):** A single flock yields continuous egg production entries over its laying cycle.

Phase 6: Logical Relational Schema Diagram (Textual Representation)

For development clarity, foreign keys point cleanly to their corresponding primary parent keys:

```
Users(UserID, Name, Email, PasswordHash, Phone, Address, Role, CreatedAt)
```

```
Flocks(FlockID, Breed, HatchDate, InitialCount, CurrentCount, Status)
```

```
EggLogs(LogID, FlockID [FK -> Flocks.FlockID], LogDate, GoodQuantity, DamagedQuantity)
```

```
Products(ProductID, ProductName, Category, UnitPrice, StockQuantity, Description)
```

```
Orders(OrderID, CustomerID [FK -> Users.UserID], OrderDate, TotalAmount, Status)
```

```
OrderItems(OrderItemID, OrderID [FK -> Orders.OrderID], ProductID [FK -> Products.ProductID],  
Quantity, Subtotal)
```

Phase 7: Physical Database Implementation & Population (DDL & DML)

Below is the optimized schema construction query blueprint to execute directly on the MySQL server environment:

```
-- Database Initialization
CREATE DATABASE IF NOT EXISTS featherflow_db;
USE featherflow_db;

-- 1. Users Table
CREATE TABLE Users (
    UserID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    Phone VARCHAR(20),
    Address TEXT,
    Role VARCHAR(15) NOT NULL DEFAULT 'Customer',
    CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_role CHECK (Role IN ('Admin', 'Customer'))
);

-- 2. Flocks Table
CREATE TABLE Flocks (
    FlockID INT AUTO_INCREMENT PRIMARY KEY,
    Breed VARCHAR(50) NOT NULL,
    HatchDate DATE NOT NULL,
    InitialCount INT NOT NULL,
    CurrentCount INT NOT NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Brooding',
    CONSTRAINT chk_count CHECK (CurrentCount >= 0)
);

-- 3. Products Table
CREATE TABLE Products (
    ProductID INT AUTO_INCREMENT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Category VARCHAR(50) NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    Description TEXT,
    CONSTRAINT chk_price CHECK (UnitPrice >= 0.00)
);

-- 4. Orders Table
CREATE TABLE Orders (
    OrderID INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    TotalAmount DECIMAL(10,2) NOT NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Pending',
```

```
FOREIGN KEY (CustomerID) REFERENCES Users(UserID) ON DELETE RESTRICT
);
```

Phase 8: Web Interface Architecture & Dashboard Mockups

The web tier is designed using modular layers. The **Frontend (HTML, Tailwind CSS, Vanilla JS)** handles the interactive presentations, making async API fetch calls to the **PHP Backend API Engine**, which communicates directly via **PDO Database Drivers** to retrieve data.

Frontend / Backend Technical Scope:

- **Admin Dashboard:** A single-page control panel to (1) Register daily egg yields from layers, (2) Monitor flock counts, (3) Post new products to the shop, and (4) Track order fulfillment statuses.
- **Customer E-Commerce Front:** A responsive grid-based shopping experience to add items to cart, input checkout details, and dynamically post transactional order payloads to the backend.